



Provably Fair Audit

Catapult (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Provably Fair Audit Scope	6
Security Rating	7
Severity Definitions	8
Status Definitions	9
Provably Fair Audit Findings	10
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY PROVABLY FAIR AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Catapult team partnered with Hashlock to conduct a Provably Fair Audit of their Catapult Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Catapult Turbo is a component of the Catapult DeFi ecosystem, offering a provably-fair token launch and trading experience where users can mint and trade custom tokens whose price paths are generated using a geometric Brownian motion model, with cryptographic commitments stored on-chain to ensure fairness. The platform is designed to democratize token launches and speculative trading by removing bot-dominance and traditional liquidity requirements, while integrating revenue-sharing incentives tied to the broader Catapult protocol's token and community engagement.

Project Name: Catapult

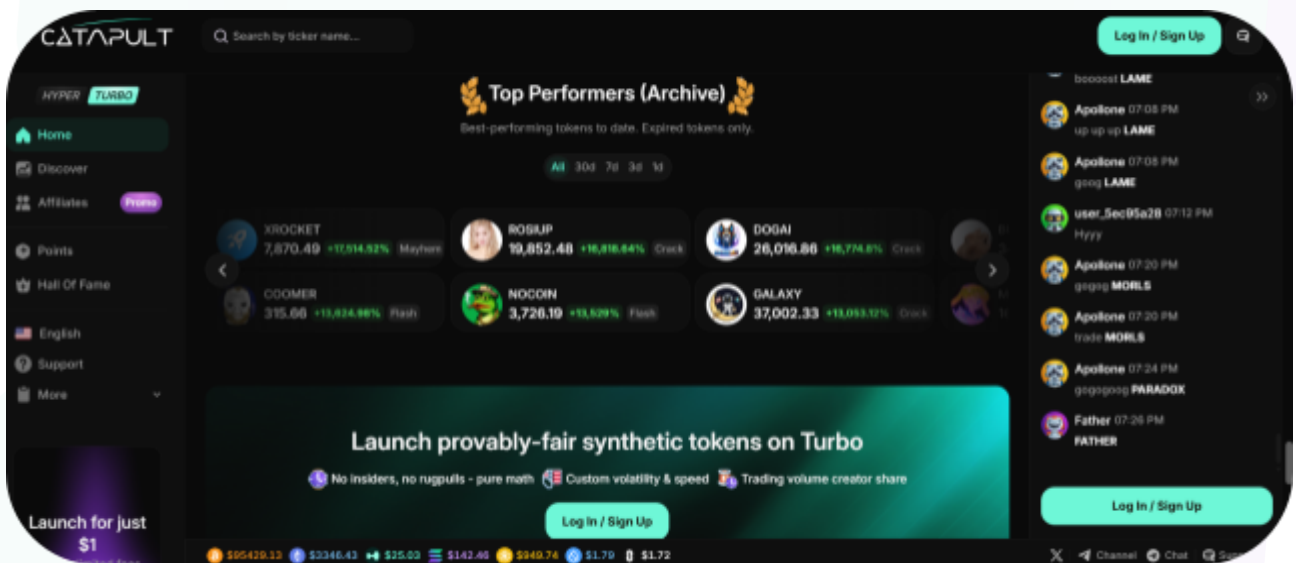
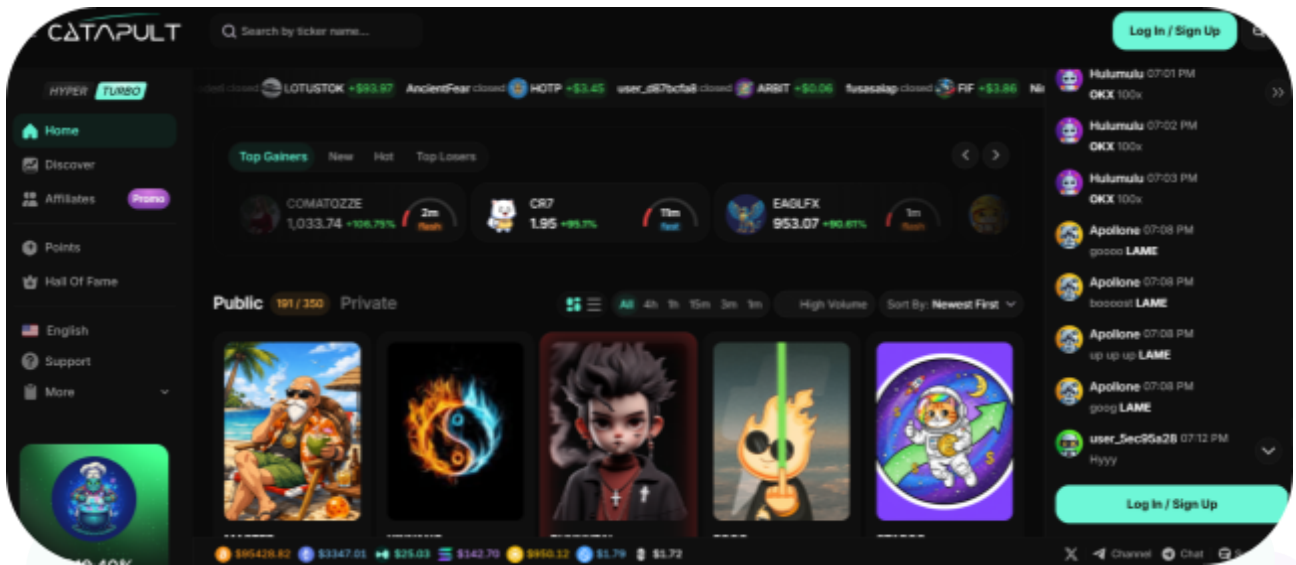
Project Type: Defi

Website: <https://catapult.trade/>

Logo:



Project Visuals:



Provably Fair Audit Scope

At Hashlock, we executed a comprehensive Provably Fair Audit on the Catapult project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Catapult Project code base
Platform	NestJS + Typescript
Provably Fair Audit Date	January, 2026
Repository/Website URL/IP Tested	https://github.com/Catapult-Org/turbo-token-generator/tree/main/src
GitHub Commit Hash (If applicable)	4bd346cf286a56205f69524868c50479d67364cb
Fix Review GitHub Commit Hash (If applicable)	b6f2b400c1c9e722f982dbd0088c28a8d14b50a6

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Provably Fair Audit, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Provably Fair Audit Findings](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

1 Medium-severity vulnerabilities

3 Low-severity vulnerabilities

1 QA

Caution: *Hashlock's Provably Fair Audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Provably Fair Audit Findings

Medium

[M-01] catapult - Public tick generation endpoint lacks rate limiting

Description

The `GeneratorController.generateToken()` endpoint is publicly accessible with no rate limiting middleware. Each request triggers `calculateTicks()` which performs CPU-intensive operations including cryptographic derivations and iterative price calculations based on the requested tick count.

Vulnerability Details

The listing below contains the affected code:

```
@Controller('generator')

export class GeneratorController {

  @Post('generate')

  generateToken(@Body() body: unknown) {

    const generateTokenRequestDto = generateTokenRequestDtoSchema.parse(body);

    return this.generatorService.generateToken(generateTokenRequestDto);

  }

}

// Each request spawns worker computation

const ticksArray = await this.turboTokenTicksCalculationService.calculateTicks({

  id: uuidv4(),

  seed,

  ticks, // User-controlled via turboTokenMode
```

```
sigma,  
  
initialPrice,  
  
speedTicksInSecond,  
  
});
```

Impact

Attackers can flood the endpoint with high-rate POST requests to exhaust CPU resources, saturate the worker thread pool, increase infrastructure costs, and degrade or deny service to legitimate users. The MAYHEM speed mode can generate 86,400 ticks requiring 86,400 cryptographic operations per request.

Recommendation

Add rate limiting middleware (e.g., [@nestjs/throttler](#)) to restrict requests per IP/user

Status

Resolved

Low

[L-01] catapult - Worker thread leaks salt value in response messages

Description

The worker thread returns the complete input object including the sensitive `salt` parameter in both success and error responses, creating information disclosure risk through logging and error handling paths.

The `ticks.worker.ts` implementation posts back the full `WorkerInputData` object (containing `salt`) as part of every response. Since `salt` is derived from the `TURBO_CHART_SALT` environment variable, any code path that logs worker results or includes them in error responses will expose this secret value

```
export type WorkerInputData = {  
  id: string;  
  seed: string;  
  ticks: number;  
  sigma: number;  
  initialPrice: number;  
  speedTicksInSecond: number;  
  salt: string; // Sensitive value  
};  
  
// Success case  
const result: TicksWorkerData = {  
  input: message, // Contains salt  
  ticks,  
};  
  
parentPort?.postMessage(result);
```

```
// Error case

parentPort?.postMessage({

  input: message, // Contains salt

  error: err as Error,

});
```

Recommendation

Modify `TicksWorkerData` to return only the `id` and essential non-sensitive fields from the input rather than echoing the complete request object. Remove `salt` from all worker messages.

Status

Resolved

[L-02] catapult - Lack of basic HTTP security hardening

Description

The application configuration does not implement security headers and CORS policies, relying solely on default Nest.js behavior. The `bootstrap()` function in `main.ts` only configures Zod exception filtering before starting the HTTP listener. No security middleware is applied to handle common HTTP-level threats or enforce security headers that protect against browser-based attacks.

Without explicit configuration, the application may:

- Bind to all network interfaces (0.0.0.0), potentially exposing the application to unwanted networks,
- Lack protection headers like CSP, X-Frame-Options, HSTS,
- Use permissive CORS defaults.

Recommendation

Implement security middleware in the bootstrap process using packages like Helmet for security headers, configure explicit CORS policies. Add explicit host binding configuration rather than relying on defaults.

Status

Resolved

[L-03] catapult - Algorithm requires Client Seed integration to meet Provably Fair standards

Description

The current implementation of the GeneratorService relies on server-side entropy (in `createNewSeed()`) and a server-side salt (`TURBO_CHART_SALT`) to generate the deterministic price charts.

While the cryptographic primitives are sound, the current design does not meet the industry definition of Provably Fair. This is because a Provably Fair system requires the user to contribute their own entropy (a `clientSeed`) to ensure the server cannot precalculate multiple seeds and cherry-pick a losing outcome before the game starts.

Recommendation

Update `GenerateTokenRequestDto` to accept an optional `clientSeed` string, pass this seed into the `seedToTicks` function in `libs/stonks.ts`, and concatenate it with the salt in `deriveSeed44HKDF` to bind the output to user input.

Status

Resolved

QA

[Q-01] catapult - Vulnerable dependencies

Description

Running an npm audit revealed that some dependencies contain known vulnerabilities. While not directly exploitable, this creates unnecessary risk:

```
# npm audit report

diff <8.0.3

jsdiff has a Denial of Service vulnerability in parsePatch and applyPatch -
https://github.com/advisories/GHSA-73rr-hh4g-fpgx

fix available via `npm audit fix --force`

Will install ts-node@1.7.1, which is a breaking change

node_modules/diff

  ts-node <=1.4.3 || >=1.7.2

  Depends on vulnerable versions of diff

node_modules/ts-node

  jest-config >=26.6.0

  Depends on vulnerable versions of ts-node

node_modules/jest-config

  @jest/core >=26.6.0

  Depends on vulnerable versions of jest-config

node_modules/@jest/core

  jest >=26.6.0

  Depends on vulnerable versions of @jest/core

  Depends on vulnerable versions of jest-cli

node_modules/jest
```

ts-jest >=27.0.0-alpha.0

Depends on vulnerable versions of jest

node_modules/ts-jest

create-jest >=29.7.0

Depends on vulnerable versions of jest-config

node_modules/create-jest

jest-cli >=26.6.0

Depends on vulnerable versions of @jest/core

Depends on vulnerable versions of create-jest

Depends on vulnerable versions of jest-config

node_modules/jest-cli

glob 10.2.0 - 10.4.5

Severity: high

glob **CLI:** Command injection via -c/--cmd executes matches with shell:true - <https://github.com/advisories/GHSA-5j98-mcp5-4vw2>

fix available via `npm audit fix`

node_modules/glob

@nestjs/cli 2.0.0-rc.1 - 10.4.9

Depends on vulnerable versions of @angular-devkit/schematics-cli

Depends on vulnerable versions of glob

Depends on vulnerable versions of inquirer

node_modules/@nestjs/cli

tmp <=0.2.3

tmp allows arbitrary temporary file / directory write via symbolic link `dir` parameter - <https://github.com/advisories/GHSA-52f5-9888-hmc6>

fix available via `npm audit fix`

node_modules/tmp



```
external-editor >=1.1.1
```

Depends on vulnerable versions of tmp

```
node_modules/external-editor
```

```
inquirer 3.0.0 - 8.2.6 || 9.0.0 - 9.3.7
```

Depends on vulnerable versions of external-editor

```
node_modules/@angular-devkit/schematics-cli/node_modules/inquirer
```

```
node_modules/inquirer
```

```
@angular-devkit/schematics-cli 0.12.0-beta.0 - 18.1.0-rc.1
```

Depends on vulnerable versions of inquirer

```
node_modules/@angular-devkit/schematics-cli
```

14 vulnerabilities (12 low, 2 high)

Recommendation

It is recommended to update all dependencies to secure versions.

Status

Resolved

Conclusion

After Hashlock's analysis, the Catapult project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Provably Fair Audits a collaborative effort. The objective of our security Provably Fair Audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Provably Fair Audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Provably Fair Audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box Provably Fair Auditing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Provably Fair Audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Provably Fair Audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Provably Fair Audit can't guarantee the explicit security of the Provably Fair Audited code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.